

uC/OS-II 即時系統專題報告

主題: RM、EDF 與 PCP 實作

B1229014 張宇宏, B1229018 洪崑誌, B1229047 陳碩軒, B1229057 陳泓瑜

1. 專題概述

本專題以 uC/OS-II 為基礎, 實作 Rate Monotonic (RM)、Earliest Deadline First (EDF) 與 Priority Ceiling Protocol (PCP)。透過修改核心排程器、TCB 結構與 Semaphore 機制, 使系統具備週期性任務管理、動態 Deadline 排程以及優先權封頂功能。

2. 實作細節

1. EDF 排程核心: 排程邏輯實作在共用的 OS_CORE.C, 以 `#ifdef SCHED_EDF` 包住 `OS_Sched()` 在每次排程時, 掃描所有 ready 任務, 把絕對 deadline 最小者動態提升為 priority 1 來執行。應用任務一律以靜態優先權 `BASE_PRIO(10)` 以上建立, 真正的執行順序完全由 deadline 決定, 符合 EDF「最早截止優先」的定義。
2. 週期任務模型與 taskset: 任務參數從 `taskset.txt` 讀入 (`exec`、`period` 以秒為單位, 乘上 `OS_TICKS_PER_SEC = 200` 換成 tick)。採隱含期限模型, 相對 `deadline = period`。每個任務建立時把絕對 deadline 錨定為 `MyStartTime + period`, 之後每完成一個週期 `+= period`。
3. CPU 時間計量: `OSTCBBRunCnt`: kernel 在 `OSTimeTick()` 每個 tick 只幫「當下正在跑」的任務 `OSTCBBRunCnt++`, 被搶佔的時間不計入。任務體用此計數器空轉: `while ((OSTCBBRunCnt - run_base) < OSTCBExecTime);` 確保被搶佔的任務仍會吃滿應有的 CPU 時間才算完成, 而非用 wall-clock 計時(那會把被搶走的时间誤算成已執行)。
4. 週期漂移修正: 絕對 deadline 錨定: 睡眠時間以 `deadline - end_tick` 計算, 而非舊版的 `period - 執行時間`。後者會把「被釋放後、開始執行前的等待」漏掉而逐期漂移; 錨定絕對 deadline 後, 釋放點精準落在週期邊界(已實測 Task2 在 6 秒)。並做了 `> 60000` 分段呼叫 `OSTimeDly`, 避開 `INT16U` 參數溢位。
5. Context Switch 紀錄: `OSTaskSwHook + ring buffer`: 切換紀錄填在原本為空的 `OSTaskSwHook()` (`OS_CPU_C.C`), 寫入全域環狀緩衝 `CtxLog[2048]`, 每筆 {time, from_id, to_id}。為符合 ISR/臨界區安全, 只做陣列寫入、直接讀全域 `OSTime` (不呼叫會巢狀臨界區的 `OSTimeGet()`)。任務識別用穩定的 `OSTCBId` (建立時設為 `i+1`), 與 EDF 動態優先權脫鉤, 避免 `prio` 被臨時改成 1 而對不上。
6. 時間流分段顯示: `TaskStart` 事後讀 ring buffer 渲染: 每一段連續 CPU 執行 = 一行 (`TaskN: start=..s end=..s`), 由上往下堆疊 (第 8-23 列、滿了環狀回捲)。以 `seg_start_tick[]` 記每個任務的段起點, 切出時收尾、切入時開新段, 並合併穿越 `TaskStart(prio 0)` 的系統開銷。顯示時間一律減 `MyStartTime` 從 0 起算, 清楚呈現任務被搶佔切成多段的時間軸。

3. RM 排程實作

讀取 taskset.txt 後依 period 排序, period 越短者配置越高優先權。PeriodicTask() 使用 OSTimeGet() 計算執行時間, 完成後以 OSTimeDly() 等待下一週期。

4. EDF 排程實作

於 OSIntExit() 中搜尋所有 Ready Task, 選出 Deadline 最早者, 透過 OSTaskChangePrio() 動態提升至最高優先權, 以模擬 EDF 行為。任務每完成一個週期後更新 Deadline。

5. PCP 實作

擴充 Semaphore 結構加入 OSEventCeiling 與 OSEventOwner。當高優先權任務被阻擋時, 持有資源的任務會被提升至 Ceiling Priority, 避免 Priority Inversion。

6. 測試與問題修正

完成 RM、EDF 與 PCP 測試案例, 並修正 OSTaskCreateExt 介面變更、Semaphore Priority Handoff、taskset.txt 路徑以及顯示時間同步等問題。

問題一: Task 結束時間相同

問題: 部分 Task 應等待 Semaphore 釋放後才能執行, 但系統卻出現多個 Task 同時結束的情況。

修正: 將 Execution Time 的計算改為在取得 Semaphore 後才開始計時, 使執行時間反映實際運行狀況。

問題二: Semaphore 等待順序錯誤

問題: Task3 釋放 Semaphore 後, 等待中的 Task2 先於優先權較高的 Task1 執行, 違反 PCP 預期行為。

修正: 在 OSSemPend() 中保留原有 Wait List 機制, 不再修改等待 Task 的 Priority; 於 OSSemPost() 使用 OS_EventTaskRdy() 喚醒最高優先權等待者, 確保 Task1 先於 Task2 執行。

問題三: Task2 #3 消失

問題: Task2 第二次執行結束時, 因 Tick 判斷誤差被誤認為錯過下一次 Release, 導致 Task2 #3 未被執行。

修正: 調整 Release 判定邏輯, 若 Release 與 End 位於同一秒則視為正常執行, 不直接跳到下一週期。

問題四: Task1 執行編號錯誤

問題: Task1 的第二次執行被誤判, 導致畫面直接顯示為 Task1 #3。

修正: 改為取得 Semaphore 並記錄實際 start_tick 後, 再更新 release_tick 與 run_count, 確保執行編號正確。

問題五:EDF 優先權未正確恢復

問題:EDF 排程中, Deadline 最早的 Task 被提升為 Priority 1 後, 完成執行時未恢復原始 Priority, 導致多個 Task 同時具有最高優先權。

修正:在 EDF Scheduler 中加入 Priority 還原機制, 當 Task 不再具有最早 Deadline 時, 恢復其原始 Priority, 確保 EDF 能正確選出下一個最緊急任務。

執行結果展示(請貼上螢幕截圖)

RM 執行結果:

```
C:\n
命令提示字元 - TEST.EXE
uC/OS-II  --  RM Scheduler  --  OS Pr
Scheduler: Rate Monotonic <RM>  !  shorter per
Task Config:
T0:e=1s,p=5s  T1:e=5s,p=10s
--- Context Switch Trace: each line = one CPU
Task0:  start= 0s  end= 1s
Task1:  start= 1s  end= 5s  <preempted>
Task0:  start= 5s  end= 6s
Task1:  start= 6s  end= 7s
Task0:  start= 10s end= 11s
<-- PRESS ESC TO QUIT
```

EDF 執行結果:

```
C:\ 命令提示字元 - TEST.EXE
uC/OS-II -- EDF Scheduler -- OS Practice Final Project
Scheduler: Earliest Deadline First (EDF) !
Task Config:
T1:e=1s,p=4s,d=4s  T2:e=5s,p=10s,d=10s
--- Context Switch Trace: each line = one CPU segment (seconds) ---
Task0: start= 0s end= 1s
Task1: start= 1s end= 4s <preempted>
Task0: start= 4s end= 5s
Task1: start= 5s end= 7s
Task0: start= 8s end= 9s
Task1: start= 10s end= 12s <preempted>
Task0: start= 12s end= 13s
Task1: start= 13s end= 16s
```

PCP 執行結果:

```
C:\ 命令提示字元 - TEST.EXE
uC/OS-II -- RM + PCP (Fixed Handoff) -- OS Practice Final Project
Scheduler: RM ! Bonus: Priority Ceiling Protocol on shared resource
Task Config:
T1:e=1s,p=4s,sem=Y  T2:e=1s,p=5s,sem=N  T3:e=5s,p=20s,sem=Y
--- Context Switch Trace: each line = one CPU segment (seconds) ---
Task1: start= 0s end= 1s
Task2: start= 1s end= 2s
Task3: start= 2s end= 4s <preempted>
Task3: start= 4s end= 7s <preempted>
Task1: start= 7s end= 8s
Task2: start= 8s end= 9s
Task2: start= 10s end= 11s
Task1: start= 12s end= 13s
```